

Penerapan Praktik PHP

Modul Intermediate Programmer

Pemetaan 12 bab modul *Training Programmer — PHP & Laravel v2* terhadap implementasi nyata di project Inventory & Order Management System, disertai nama file dan nomor baris sebagai bukti. IOMS dibangun PHP native tanpa framework, sehingga bab-bab bertema Laravel dipetakan ke pendekatan native yang setara.

PROJECT

IOMS (ioms/)

DISUSUN OLEH

Arief Aryudi Syidik

REFERENSI MODUL

PHP & Laravel v2, Edisi 2.0

TANGGAL

25 September 2026

- ✓ Diterapkan
- Sebagian
- ✗ Belum / Tidak relevan

BAB	TOPIK	STATUS
01	PHP values, operators, dan runtime configuration	Diterapkan
02	Function contracts, scope, dan callables	Diterapkan
03	Strings, patterns, dan output formatting	Diterapkan
04	Arrays dan data transformation	Diterapkan
05	Object contracts dan lifecycle	Diterapkan
06	Secure PHP boundaries	Diterapkan
07	Laravel routing, controllers, dan API	Sebagian — native equivalent

BAB	TOPIK	STATUS
08	Migrations, Eloquent, dan query quality	Sebagian — native equivalent
09	Middleware, authorization, dan file pipelines	Sebagian — native equivalent
10	Errors, observability, dan automated testing	Diterapkan
11	SOLID dan design patterns	Diterapkan
12	Refactoring, resource optimization, dan mini-project	Diterapkan

01 PHP Values, Operators, dan Runtime Configuration

Diterapkan

Type dijaga eksplisit lewat `declare(strict_types=1)` di seluruh file, casting eksplisit di boundary data (row database, input HTTP), dan konfigurasi runtime dibaca lewat satu titik terpusat.

Seluruh 73 file di app/ (100%)

`declare(strict_types=1);` dipasang di baris pertama tiap file — PHP tidak melakukan coercion implisit antar tipe skalar pada seluruh business logic.

app/Core/Env.php :12–78

Loader environment variable tersentral: `Env::load()` parsing `.env` sekali, `Env::get(string $key, mixed $default)` memakai null-coalescing (`??`) untuk fallback, dan `Env::bool()` memakai `match(strtolower(...))` untuk konversi string ke boolean eksplisit — bukan asumsi truthy/falsy implisit.

app/Core/Database.php :23–27

Setiap nilai konfigurasi DB di-cast eksplisit (`string`) `Env::get(...)` sebelum dipakai membangun DSN PDO — kontrak tipe dijaga di boundary, bukan dibiarkan mixed.

Catatan: audit `php.ini`/extension formal (mis. daftar extension aktif di image Docker) tidak didokumentasikan sebagai artefak terpisah; konfigurasi PHP mengikuti default image `php:8.2-apache` pada `Dockerfile`.

02 Function Contracts, Scope, dan Callables

Diterapkan

Setiap method punya signature tipe parameter & return yang eksplisit; closure dipakai untuk transformasi data yang scoped dan tidak membocorkan state ke luar.

app/Core/Router.php :66

`preg_replace_callback()` dengan closure `static function (array $m) use (&$names): string` — closure statik (tidak mengikat `$this`), scope ditutup eksplisit lewat `use (&$names)`, return type dideklarasikan.

app/Core/View.php :34

Renderer template dibungkus closure statik `static function (string $__file, array $__data): string` agar variabel view tidak bocor ke scope pemanggil `include`.

app/Repository/MySQLPurchaseOrderRepository.php :93 (representatif — pola sama di semua Repository)

`array_map(fn ($r) => PurchaseOrder::fromRow($r), $stmt->fetchAll())` — arrow function sebagai callable ringkas untuk transformasi baris database menjadi Entity, dipakai konsisten di semua Repository (14+ titik).

03 Strings, Patterns, dan Output Formatting

Diterapkan

Output ke HTML selalu di-escape; pola regex dipakai untuk parsing route; nama file acak dibangun dari representasi hex.

app/Core/View.php :57, 81

`htmlspecialchars((string) $value, ENT_QUOTES, 'UTF-8')` — dipanggil lewat helper global `e()` yang dipakai di 34+ file view, mencegah XSS dari data yang berasal dari database/input user.

app/Core/Router.php :66

Pattern `#\{([a-zA-Z_][a-zA-Z0-9_]*)\}` — mem-parsing segmen `{param}` pada definisi route menjadi regex capturing group.

app/Service/ProductService.php :169

`bin2hex(random_bytes(16)) . '.' . $ext` — nama file upload dibangun dari representasi hex byte acak, bukan nama file asli (mencegah path traversal & collision).

04 Arrays dan Data Transformation

Diterapkan

`array_map/array_filter/array_values` dipakai konsisten untuk transformasi data immutable, baik di implementasi MySQL maupun In-Memory (test double).

app/Repository/InMemorySalesOrderRepository.php :52, 55, 88

Filter in-memory memakai `array_values(array_filter(...))` — re-index array setelah filter agar konsumen tidak bergantung pada key yang mungkin bolong.

app/Repository/InMemoryProductStockRepository.php :53, 66, 79

`array_map('intval', explode(':', $key))` — parsing composite key "productId:warehouseId" jadi array integer, dipakai sebagai key sintetis pengganti composite primary key di implementasi in-memory.

app/Repository/MySql*Repository.php (14 titik di seluruh Repository)

Pola konsisten `array_map(fn ($r) => Entity::fromRow($r), $stmt->fetchAll())` mengubah hasil query mentah (array asosiatif) menjadi array of Entity bertipe.

05 Object Contracts dan Lifecycle

Diterapkan

Interface sebagai contract, class `final` untuk mencegah override tak terduga, dan Exception domain sebagai objek bertipe untuk merepresentasikan failure state.

app/Repository/*Interface.php (10 interface)

10 interface Repository mendefinisikan contract akses data; 4 di antaranya (Product, ProductStock, SalesOrder, User) punya 2 implementasi (MySQL + In-Memory) — lihat ARCH-01.

app/Service/*.php, app/Repository/*.php, app/Entity/*.php (38 class)

38 class dideklarasikan `final class` — mencegah subclass tak terduga yang bisa mengubah kontrak Service/Repository/Entity secara implisit.

app/Service/Exception/ValidationException.php :9–22

Exception domain dengan *constructor property promotion* (`private array $errors`) dan method `errors()` : `array` — objek exception membawa data terstruktur, bukan sekadar string pesan. Pola sama di `AuthorizationException`, `InsufficientStockException`, `InvalidStatusTransitionException`.

06 Secure PHP Boundaries

Diterapkan

Empat threat utama modul ini (injection, XSS, unsafe upload, secret exposure) masing-masing punya mitigasi konkret di boundary yang tepat.

app/Repository/*.php (66 pemanggilan prepare/query)

Seluruh 66 query di layer Repository memakai `PDO::prepare()` dengan parameter binding — tidak ada string interpolation input user ke SQL (mencegah SQL injection).

app/Service/ProductService.php :14–15, 148–166

Upload gambar: whitelist MIME (`ALLOWED_MIME`), limit ukuran 2MB (`MAX_BYTES`), validasi `mime_content_type()` pada file asli (bukan percaya `Content-Type` header dari klien), ekstensi ditentukan dari MIME hasil deteksi — bukan dari nama file asli.

app/Core/Database.php :23–27 · docker-compose.yml :26–29

Credential database tidak hardcode — dibaca dari environment variable (`.env`, tidak di-commit; lihat `.gitignore`).

Catatan: CSRF token tidak diimplementasikan secara eksplisit (form action mengandalkan session cookie `SameSite` default browser) — di luar scope brief project, tapi diakui sebagai gap terhadap threat model modul.

07 Laravel Routing, Controllers, dan API

Sebagian — native equivalent

IOMS tidak memakai Laravel (dilarang brief), tapi konsep routing/controller/API dibangun ulang secara native dengan tanggung jawab yang setara.

app/Core/Router.php :1–30

Router native mendukung path parameter {param}, method HTTP (GET/POST), dan dispatch ke [Controller::class, 'method'] — setara routing table Laravel tanpa dependency framework.

app/Controller/ApiController.php :18–58

Endpoint GET /api/products/{sku}/availability mengembalikan JSON dengan Content-Type & kode HTTP yang tepat (200/401/404) — setara API Resource/JsonResponse Laravel.

Catatan: tidak ada HTTP client (Guzzle) karena IOMS tidak memanggil API eksternal — tidak relevan terhadap domain project ini.

08 Migrations, Eloquent, dan Query Quality

Sebagian — native equivalent

Tanpa Eloquent/migration Laravel (dilarang brief) — skema dideklarasikan lewat SQL murni dan akses data lewat Repository Pattern manual. Kualitas query (index, N+1) tetap ditegakkan.

database/schema.sql (247 baris)

Skema deklaratif dengan CREATE TABLE IF NOT EXISTS (idempotent, setara migration up()), primary/foreign key, dan CHECK constraint — berperan sebagai migration file tunggal yang portable.

app/Repository/MySqlSalesOrderRepository.php :177–184

countsByStatus(): satu query GROUP BY status menggantikan pola N+1 (dijelaskan detail di docs/report/SQL-Practice-Evidence.pdf Bab 12) — setara pencegahan N+1 Eloquent lewat with() eager loading.

database/schema.sql :60–63, 127–131

Index dipasang manual pada kolom filter/join (setara \$table->index() di migration Laravel).

09 Middleware, Authorization, dan File Pipelines

Sebagian — native equivalent

Tanpa middleware pipeline Laravel — guard otorisasi dipanggil eksplisit di awal setiap Controller method, dengan efek yang sama (blok request sebelum business logic jalan).

app/Core/Auth.php :76–107

`Auth::requireLogin()` dan `Auth::requireRole()` — dipanggil sebagai baris pertama tiap Controller method yang butuh proteksi, mengembalikan `bool` yang di-check untuk early-return. Setara middleware `auth/can`: Laravel, hanya dipanggil manual bukan lewat route pipeline.

app/Core/Auth.php :113–126

Varian `requireLoginApi()/requireRoleApi()` mengembalikan response JSON 401/403 alih-alih redirect — guard yang sadar konteks (web vs API), setara route group middleware berbeda di Laravel.

app/Service/ProductService.php :148–169

Pipeline upload file: validasi error code → validasi ukuran → validasi MIME → generate nama aman → simpan — berurutan dan setiap tahap bisa menghentikan pipeline lewat exception, setara `FormRequest` chain Laravel.

10 Errors, Observability, dan Automated Testing

Diterapkan

Exception ditangani berjenjang dengan logging terstruktur, detail teknis tidak pernah bocor ke response user, dan business logic diuji terisolasi dari infrastruktur.

app/Controller/Controller.php :75, 85, 96

catch berjenjang per jenis exception (`AuthorizationException` → 403, domain exception → 422, `Throwable` generik → 500) — masing-masing dicatat `error_log()` dengan konteks pesan + file:line asal, tapi response ke user tetap pesan generik.

public/index.php :31

`ini_set('display_errors', '0')` — stack trace PHP tidak pernah dikirim ke browser, bahkan untuk error yang tidak tertangkap oleh try-catch aplikasi.

tests/Unit/ (22 test, 4 file)

22 unit test method mencakup ≥4 area logic (validasi tanggal PO, transisi status SO, kalkulasi low-stock, otorisasi approve) — dijalankan tanpa session PHP atau koneksi MySQL nyata (pakai SQLite in-memory + Repository fake).

tests/Integration/ (4 test, 3 file)

Integration test menyentuh MySQL nyata di Docker — membuktikan goods receipt menambah stok end-to-end dan goods issue kedua ditolak saat stok habis.

11 SOLID dan Design Patterns

Diterapkan

Repository Pattern untuk Dependency Inversion, audit SRP tertulis terhadap kode nyata, dan state machine terpusat untuk menghindari duplikasi conditional.

docs/architecture/adr-001-repository-pattern.md (35 baris)

ADR menjelaskan keputusan DIP: Service bergantung pada interface Repository, bukan implementasi MySQL konkret — alasan, alternatif yang ditolak, dan konsekuensinya didokumentasikan.

docs/quality/audit-srp.md (31 baris)

Audit SRP terhadap satu class nyata yang melanggar (draft awal SalesOrderService), dengan rencana pemecahan tanggung jawab.

app/Service/SalesOrderService.php :22–28

Konstanta TRANSITIONS (state machine map) menggantikan rangkaian if/elseif tersebar — pola State/Strategy sederhana yang membuat aturan transisi status terpusat dan bisa dibaca sekali (lihat docs/quality/refactor-log.md entri #1).

12 Refactoring, Resource Optimization, dan Mini-Project

Diterapkan

Ekspor CSV memakai stream sementara (bukan menumpuk seluruh string di memori), dan query dashboard dikonsolidasi untuk mengurangi jumlah round-trip database.

app/Service/ReportService.php :32–39, 49–71

fopen('php://temp', 'r+') + fputcsv() per baris + stream_get_contents() di akhir — CSV dibangun lewat stream sementara, bukan concatenation string manual yang menumpuk seluruh isi file di satu variabel besar sebelum siap dikirim.

app/Service/DashboardService.php :34–35

Konsolidasi 8 query COUNT terpisah menjadi 2 query GROUP BY — didokumentasikan lengkap smell → teknik → before/after di docs/quality/refactor-log.md entri #4, termasuk hasil verifikasi before/after.

docs/quality/refactor-log.md (4 entri)

4 entri refactor terdokumentasi dengan format smell/teknik/before-after, termasuk minimal 1 commit bertag refactor: yang memperbaiki kode lama (Boy Scout Rule) — bukan fitur baru.